

AWS Security & Encryption

KMS, Encryption SDK, SSM Parameter Store

Why encryption?

Encryption in flight (TLS / SSL)

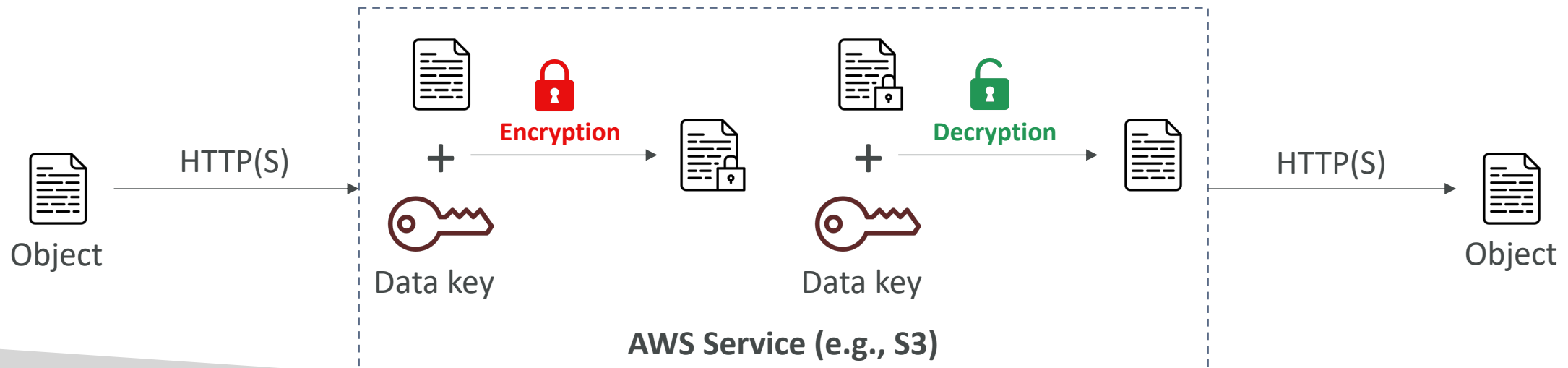
- Data is encrypted before sending and decrypted after receiving
- TLS certificates help with encryption (HTTPS)
- Encryption in flight ensures no MITM (man in the middle attack) can happen



Why encryption?

Server-side encryption at rest

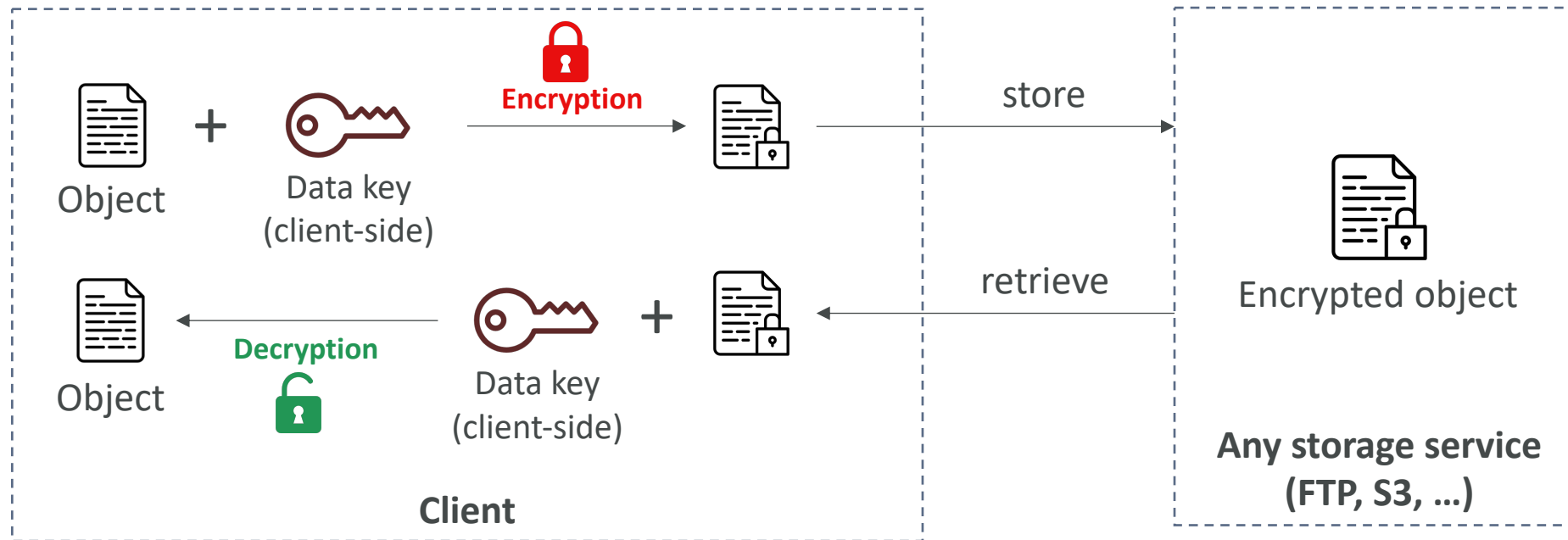
- Data is encrypted after being received by the server
- Data is decrypted before being sent
- It is stored in an encrypted form thanks to a key (usually a data key)
- The encryption / decryption keys must be managed somewhere, and the server must have access to it



Why encryption?

Client-side encryption

- Data is encrypted by the client and never decrypted by the server
- Data will be decrypted by a receiving client
- The server should not be able to decrypt the data
- Could leverage Envelope Encryption





AWS KMS (Key Management Service)

- Anytime you hear “encryption” for an AWS service, it’s most likely KMS
- AWS manages encryption keys for us
- Fully integrated with IAM for authorization
- Easy way to control access to your data
- Able to audit KMS Key usage using CloudTrail
- Seamlessly integrated into most AWS services (EBS, S3, RDS, SSM...)
- **Never ever store your secrets in plaintext, especially in your code!**
 - KMS Key Encryption also available through API calls (SDK, CLI)
 - Encrypted secrets can be stored in the code / environment variables

KMS Keys Types

- KMS Keys is the new name of KMS Customer *Master Key*
- Symmetric (AES-256 keys)
 - Single encryption key that is used to Encrypt and Decrypt
 - AWS services that are integrated with KMS use Symmetric CMKs
 - You never get access to the KMS Key unencrypted (must call KMS API to use)
- Asymmetric (RSA & ECC key pairs)
 - Public (Encrypt) and Private Key (Decrypt) pair
 - Used for Encrypt/Decrypt, or Sign/Verify operations
 - The public key is downloadable, but you can't access the Private Key unencrypted
 - Use case: encryption outside of AWS by users who can't call the KMS API

AWS KMS (Key Management Service)



- Types of KMS Keys:
 - AWS Owned Keys (free): SSE-S3, SSE-SQS, SSE-DDB (default key)
 - AWS Managed Key: **free** (aws/service-name, example: aws/rds or aws/ebs)
 - Customer managed keys created in KMS: **\$1 / month**
 - Customer managed keys imported: **\$1 / month**
 - + pay for API call to KMS (\$0.03 / 10000 calls)

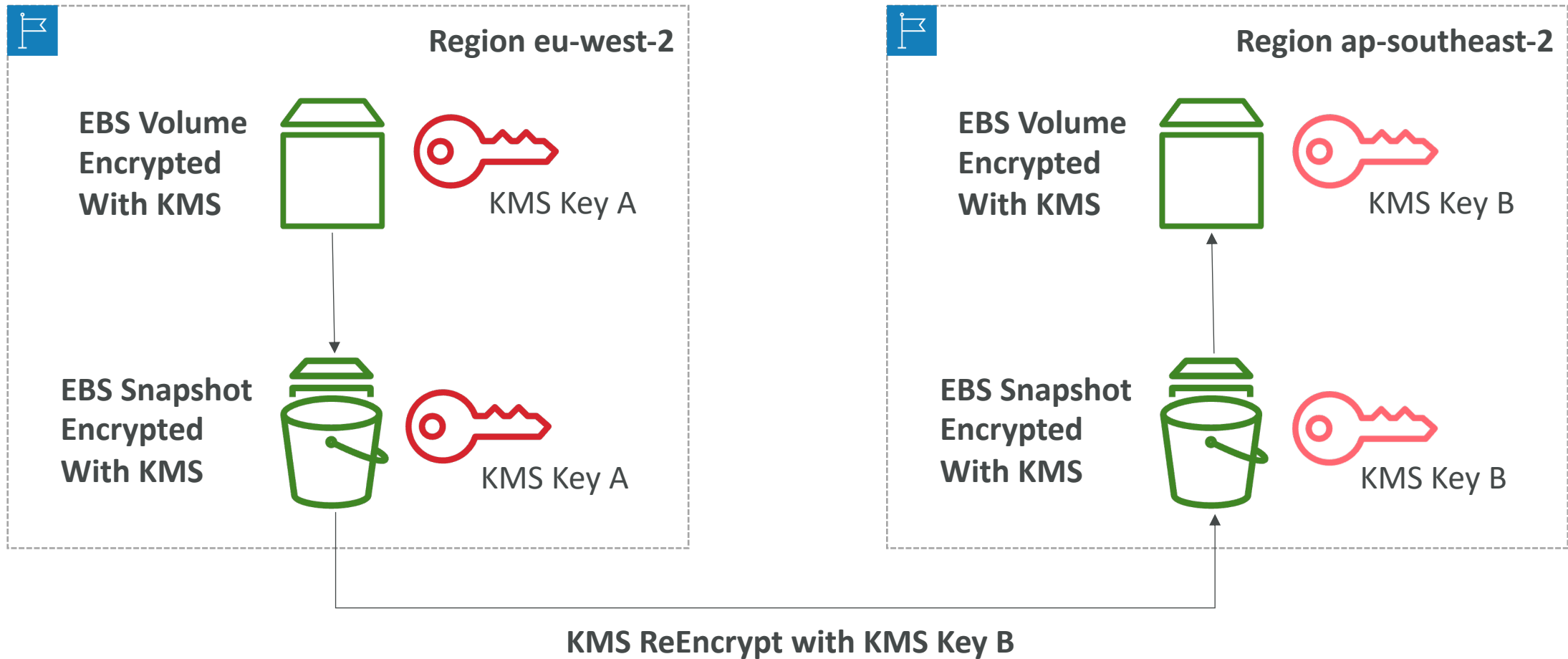


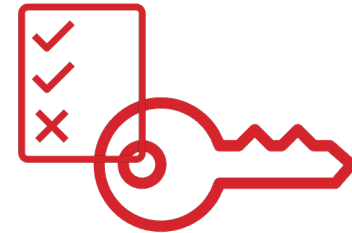
- Automatic Key rotation:
 - AWS-managed KMS Key: automatic every 1 year
 - Customer-managed KMS Key: (must be enabled) automatic & on-demand
 - Imported KMS Key: only manual rotation possible using alias

Encryption key management

- ☒ Owned by Amazon DynamoDB
- ☐ AWS managed key [Lea](#)
Key alias: aws/dynamodb.
- ☐ Stored in your account,
and owned and managed by you

Copying Snapshots across regions





KMS Key Policies

- Control access to KMS keys, “similar” to S3 bucket policies
- Difference: you cannot control access without them
- **Default KMS Key Policy:**
 - Created if you don't provide a specific KMS Key Policy
 - Complete access to the key to the root user = entire AWS account
- **Custom KMS Key Policy:**
 - Define users, roles that can access the KMS key
 - Define who can administer the key
 - Useful for cross-account access of your KMS key

Copying Snapshots across accounts

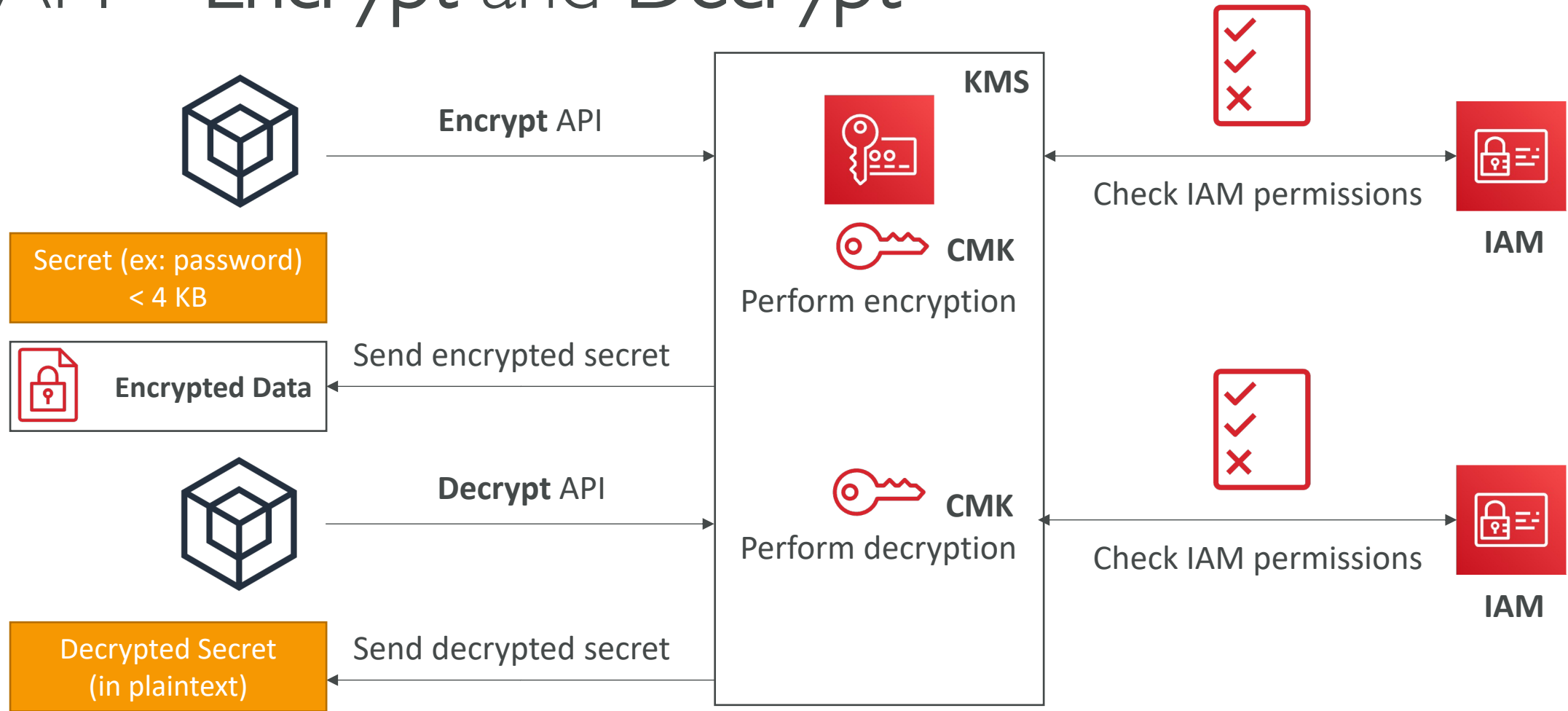
1. Create a Snapshot, encrypted with your own KMS Key (Customer Managed Key)
2. Attach a KMS Key Policy to authorize cross-account access
3. Share the encrypted snapshot
4. (in target) Create a copy of the Snapshot, encrypt it with a CMK in your account
5. Create a volume from the snapshot

```
{  
  "Sid": "Allow use of the key with destination account",  
  "Effect": "Allow",  
  "Principal": {  
    "AWS": "arn:aws:iam::TARGET-ACCOUNT-ID:role/ROLENAME"  
  },  
  "Action": [  
    "kms:Decrypt",  
    "kms:CreateGrant"  
  ],  
  "Resource": "*",  
  "Condition": {  
    "StringEquals": {  
      "kms:ViaService": "ec2.REGION.amazonaws.com",  
      "kms:CallerAccount": "TARGET-ACCOUNT-ID"  
    }  
  }  
}
```

KMS Key Policy

How does KMS work?

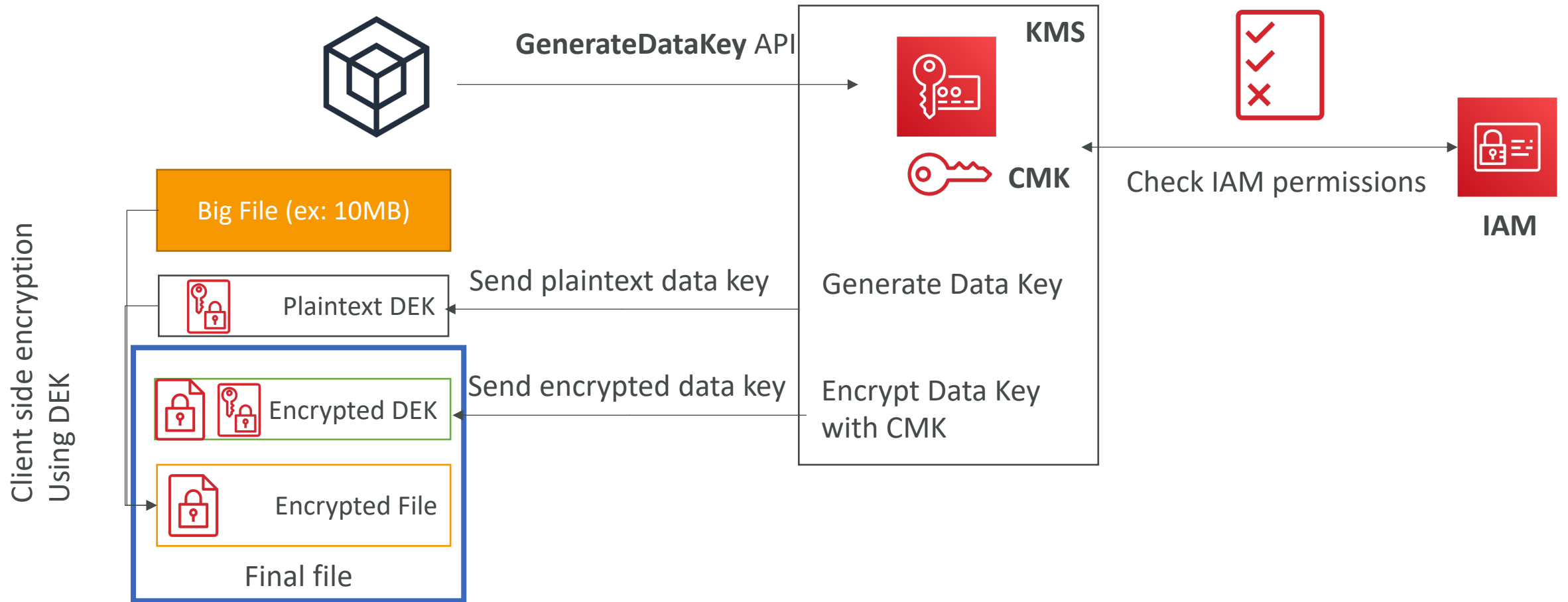
API – Encrypt and Decrypt



Envelope Encryption

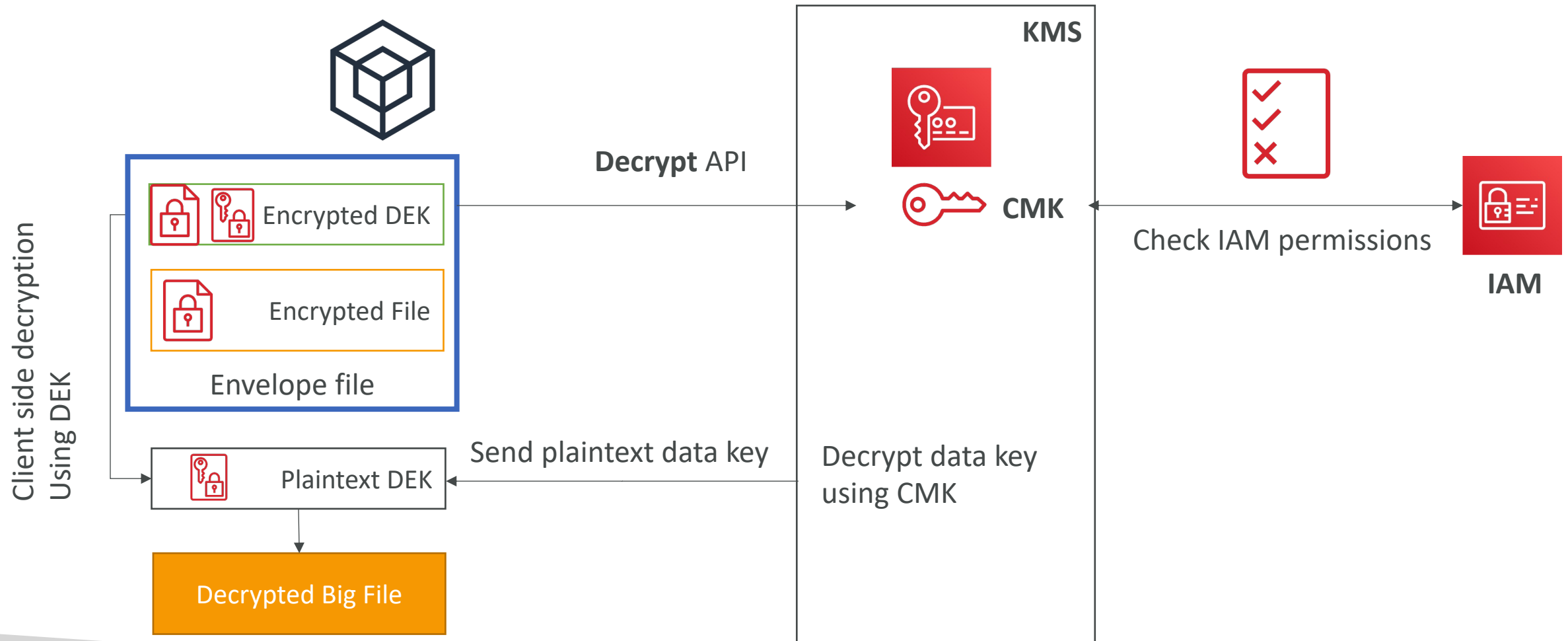
- KMS Encrypt API call has a limit of 4 KB
 - If you want to encrypt >4 KB, we need to use Envelope Encryption
 - The main API that will help us is the **GenerateDataKey** API
-
- For the exam: anything over 4 KB of data that needs to be encrypted must use the Envelope Encryption == **GenerateDataKey** API

Deep dive into Envelope Encryption GenerateDataKey API



Deep dive into Envelope Encryption

Decrypt envelope data





Encryption SDK

- The AWS Encryption SDK implemented Envelope Encryption for us
- The Encryption SDK also exists as a CLI tool we can install
- Implementations for Java, Python, C, JavaScript
- **Feature - Data Key Caching:**
 - re-use data keys instead of creating new ones for each encryption
 - Helps with reducing the number of calls to KMS with a security trade-off
 - Use LocalCryptoMaterialsCache (max age, max bytes, max number of messages)

Encryption SDK – diagram

- The SDK encrypts the data encryption key and stores it (encrypted) as part of the returned ciphertext.



KMS Symmetric – API Summary

- **Encrypt:** encrypt up to 4 KB of data through KMS
- **GenerateDataKey:** generates a unique symmetric data key (DEK)
 - returns a plaintext copy of the data key
 - AND a copy that is encrypted under the CMK that you specify
- **GenerateDataKeyWithoutPlaintext:**
 - Generate a DEK to use at some point (not immediately)
 - DEK that is encrypted under the CMK that you specify (must use Decrypt later)
- **Decrypt:** decrypt up to 4 KB of data (including Data Encryption Keys)
- **GenerateRandom:** Returns a random byte string

KMS Request Quotas

- When you exceed a request quota, you get a **ThrottlingException**:

```
You have exceeded the rate at which you may call KMS. Reduce the frequency of your calls.  
(Service: AWSKMS; Status Code: 400; Error Code: ThrottlingException; Request ID: <ID>)
```

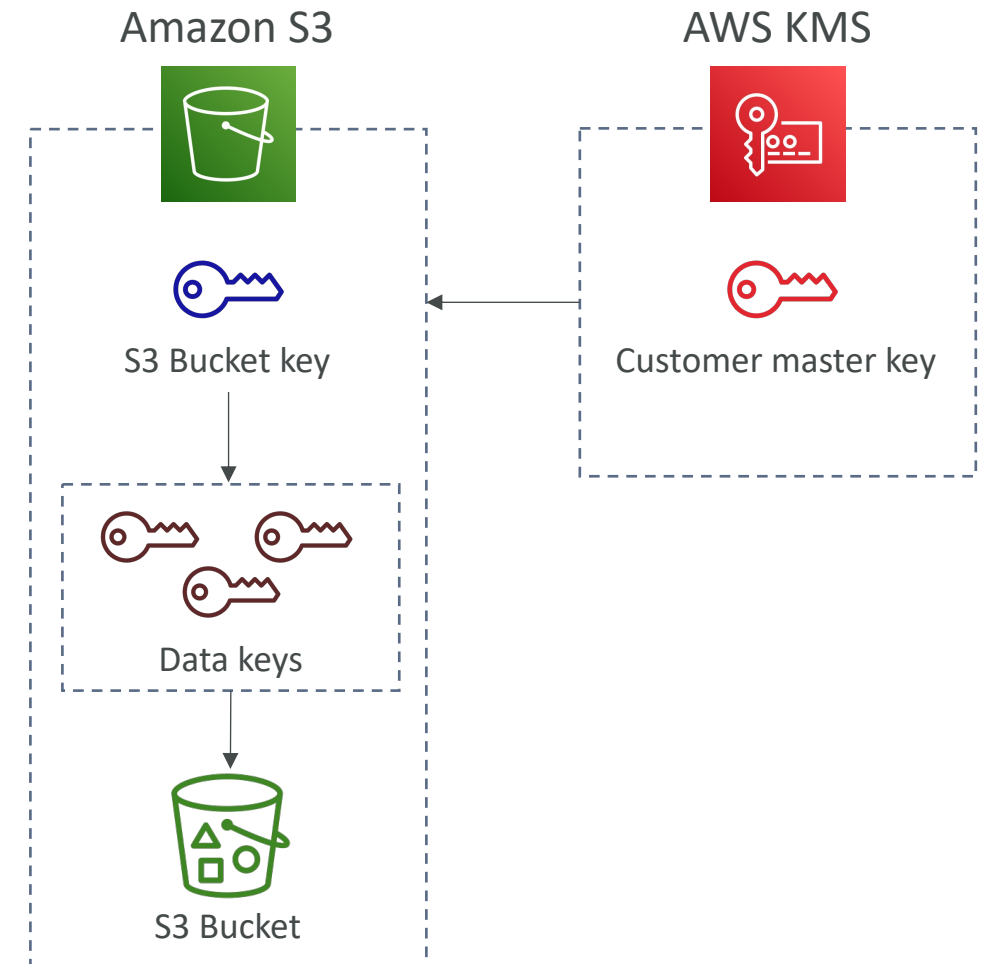
- To respond, use **exponential backoff** (backoff and retry)
- For cryptographic operations, they share a quota
- This includes requests made by AWS on your behalf (ex: SSE-KMS)
- For GenerateDataKey, consider using DEK caching from the Encryption SDK
- You can request a Request Quotas increase through API or AWS support

KMS Request Quotas

API operation	Request quotas (per second)
Decrypt Encrypt GenerateDataKey (symmetric) GenerateDataKeyWithoutPlaintext (symmetric) GenerateRandom ReEncrypt Sign (asymmetric) Verify (asymmetric)	These shared quotas vary with the AWS Region and the type of CMK used in the request. Each quota is calculated separately. Symmetric CMK quota: • 5,500 (shared) • 10,000 (shared) in the following Regions: • us-east-2, ap-southeast-1, ap-southeast-2, ap-northeast-1, eu-central-1, eu-west-2 • 30,000 (shared) in the following Regions: • us-east-1, us-west-2, eu-west-1 Asymmetric CMK quota: • 500 (shared) for RSA CMKs • 300 (shared) for Elliptic curve (ECC) CMKs

S3 Bucket Key for SSE-KMS encryption

- New setting to decrease...
 - Number of API calls made to KMS from S3 by 99%
 - Costs of overall KMS encryption with Amazon S3 by 99%
- This leverages data keys
 - A “S3 bucket key” is generated
 - That key is used to encrypt KMS objects with new data keys
- You will see **less KMS CloudTrail events in CloudTrail**



Key Policy – Examples



Default KMS Key Policy

```
{
  "Effect": "Allow",
  "Action": "kms:*",
  "Principal": {
    "AWS": "arn:aws:iam::123456789012:root"
  },
  "Resource": "*"
}
```

Allow Federated User

```
{
  "Effect": "Allow",
  "Action": [
    "kms:Encrypt",
    "kms:Decrypt",
    "kms:ReEncrypt*",
    "kms:GenerateDataKey*",
    "kms:DescribeKey"
  ],
  "Principal": {
    "AWS": "arn:aws:sts::123456789012:federated-user/user-name"
  },
  "Resource": "*"
}
```

Principal Options in IAM Policies

- AWS Account and Root User

```
"Principal": { "AWS": "123456789012" }
```

```
"Principal": { "AWS": "arn:aws:iam::123456789012:root" }
```

- IAM Roles

```
"Principal": { "AWS": "arn:aws:iam::123456789012:role/role-name" }
```

- IAM Role Sessions

```
"Principal": { "AWS": "arn:aws:sts::123456789012:assumed-role/role-name/role-session-name" }
```

```
"Principal": { "Federated": "cognito-identity.amazonaws.com" }
```

```
"Principal": { "Federated": "arn:aws:iam::123456789012:saml-provider/provider-name" }
```

Principal Options in IAM Policies

- IAM Users

```
"Principal": { "AWS": "arn:aws:iam::123456789012:user/user-name" }
```

- Federated User Sessions

```
"Principal": { "AWS": "arn:aws:sts::123456789012:federated-user/user-name" }
```

- AWS Services

```
"Principal": {  
  "Service": [  
    "ecs.amazonaws.com",  
    "elasticloadbalancing.amazonaws.com"  
  ]  
}
```

- All Principals

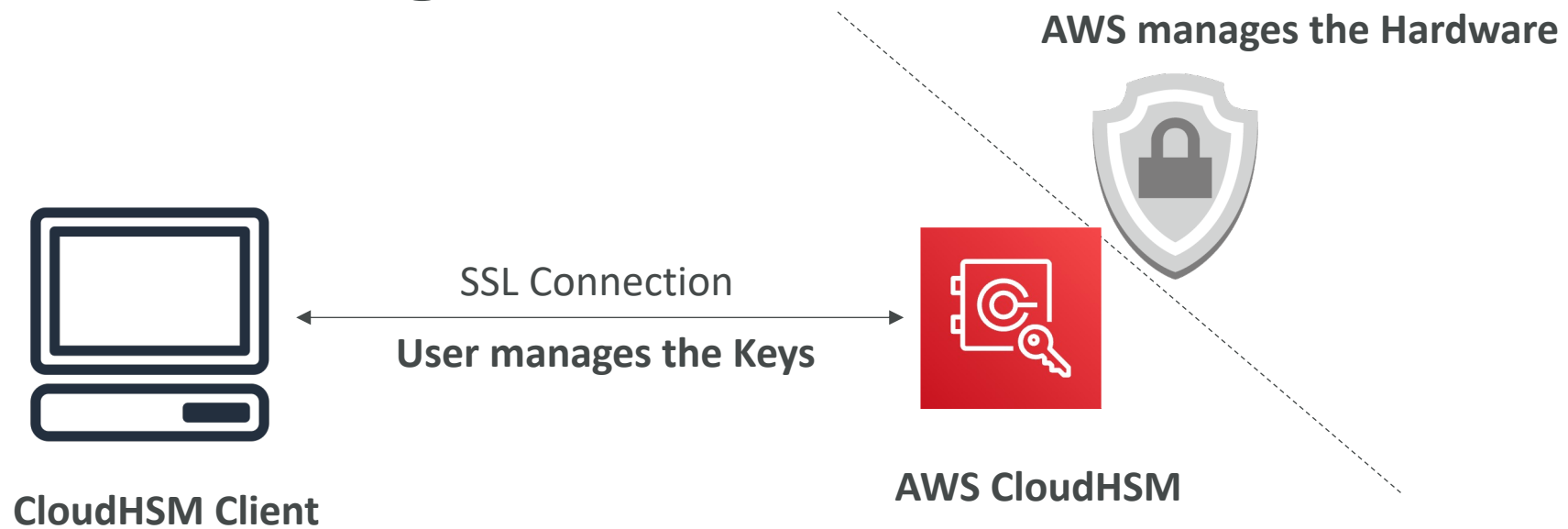
```
"Principal": "*"  
"Principal": { "AWS": "*" }
```



CloudHSM

- KMS => AWS manages the software for encryption
- CloudHSM => AWS provisions encryption **hardware**
- Dedicated Hardware (HSM = Hardware Security Module)
- You manage your own encryption keys entirely (not AWS)
- HSM device is tamper resistant, FIPS 140-2 Level 3 compliance
- Supports both symmetric and **asymmetric** encryption (SSL/TLS keys)
- No free tier available
- Must use the CloudHSM Client Software
- Redshift supports CloudHSM for database encryption and key management
- Good option to use with SSE-C encryption

CloudHSM Diagram



IAM permissions:

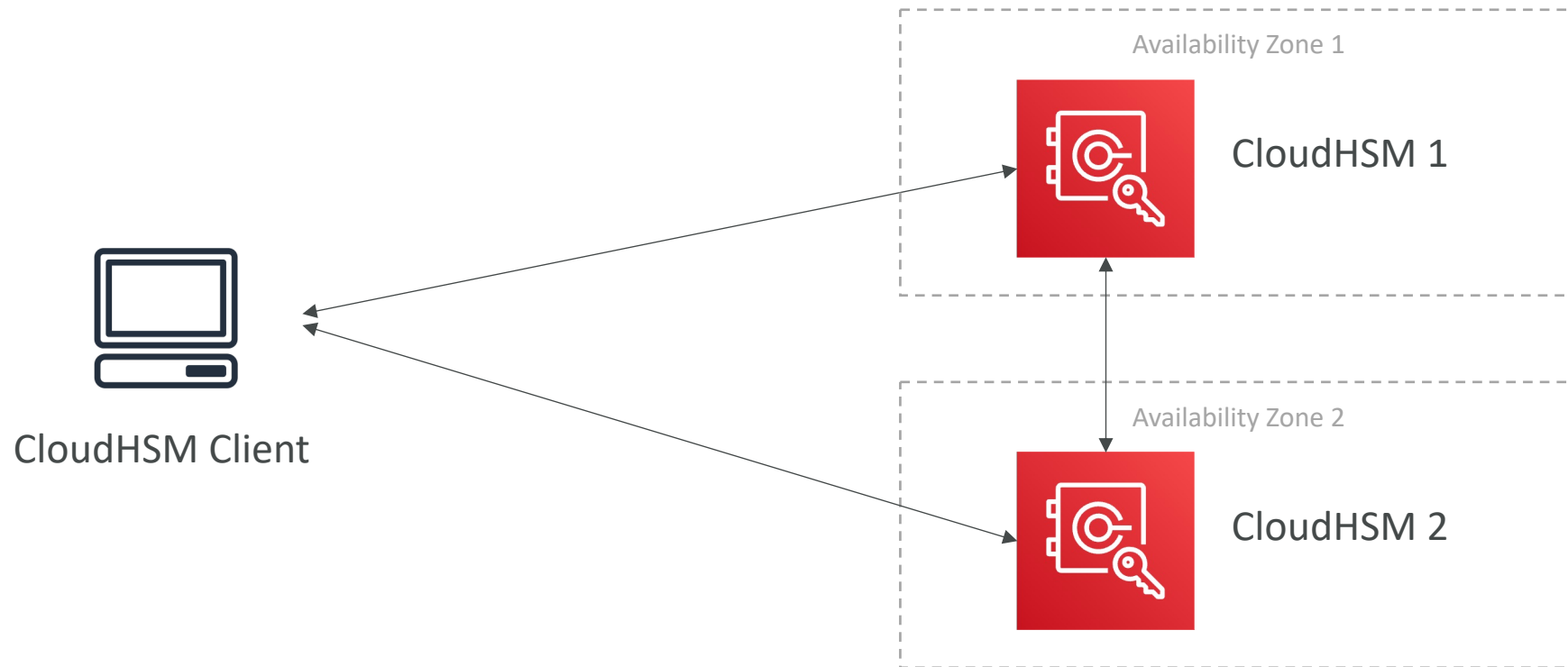
- CRUD an HSM Cluster

CloudHSM Software:

- Manage the Keys
- Manage the Users

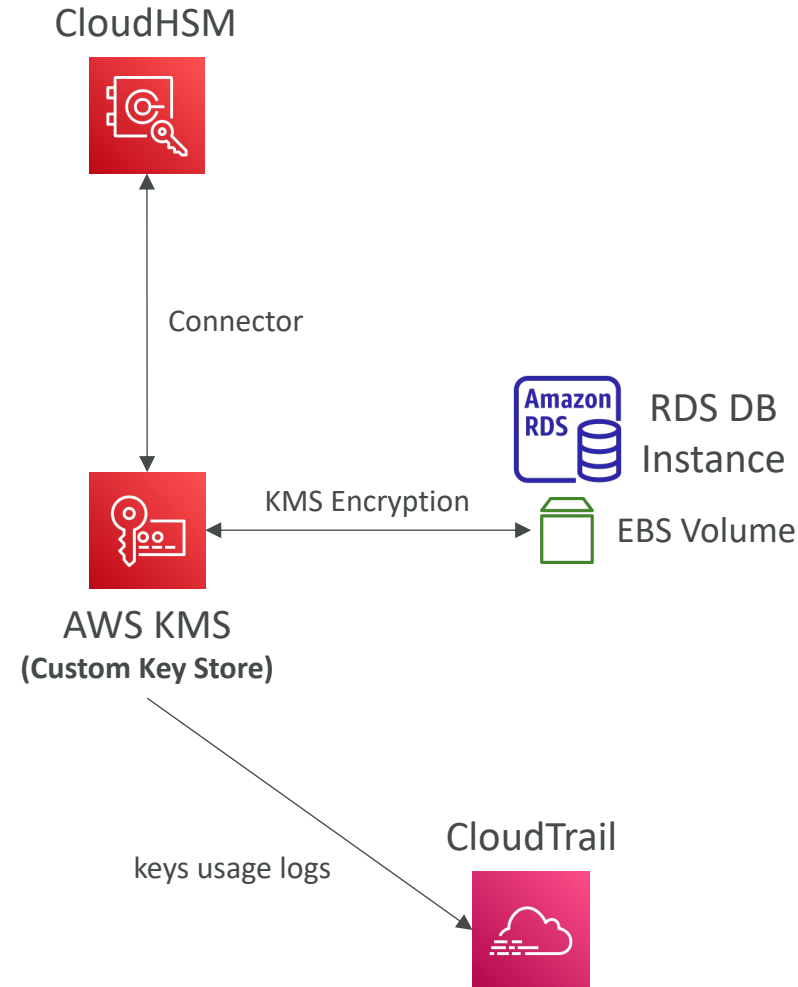
CloudHSM – High Availability

- CloudHSM clusters are spread across Multi AZ (HA)
- Great for availability and durability



CloudHSM – Integration with AWS Services

- Through integration with AWS KMS
- Configure KMS Custom Key Store with CloudHSM
- Example: EBS, S3, RDS ...



CloudHSM vs. KMS

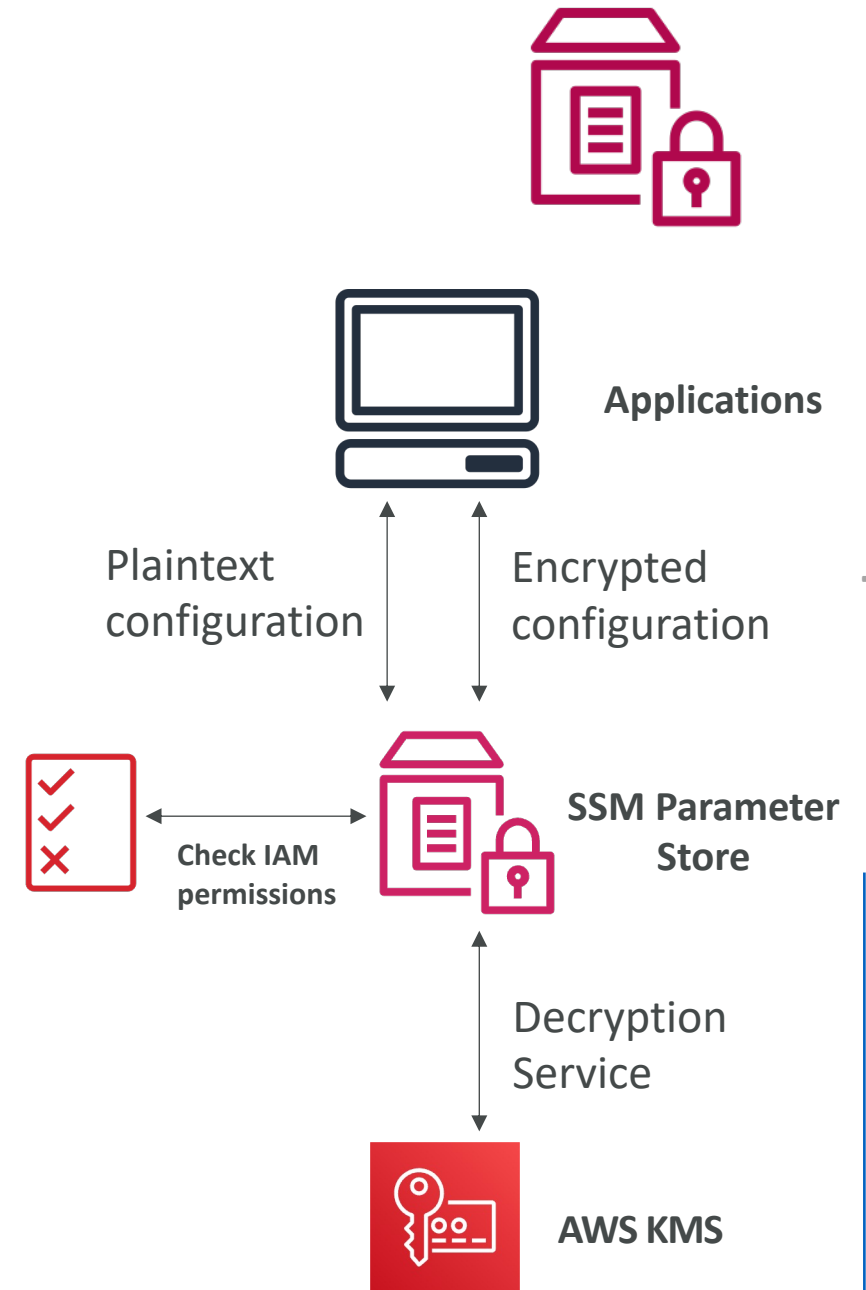
Feature	AWS KMS	AWS CloudHSM
Tenancy	Multi-Tenant	Single-Tenant
Standard	FIPS 140-2 Level 3	FIPS 140-2 Level 3
Master Keys	• AWS Owned CMK • AWS Managed CMK • Customer Managed CMK	Customer Managed CMK
Key Types	• Symmetric • Asymmetric • Digital Signing	• Symmetric • Asymmetric • Digital Signing & Hashing
Key Accessibility	Accessible in multiple AWS regions (can't access keys outside the region it's created in)	• Deployed and managed in a VPC • Can be shared across VPCs (VPC Peering)
Cryptographic Acceleration	None	• SSL/TLS Acceleration • Oracle TDE Acceleration
Access & Authentication	AWS IAM	You create users and manage their permissions

CloudHSM vs. KMS

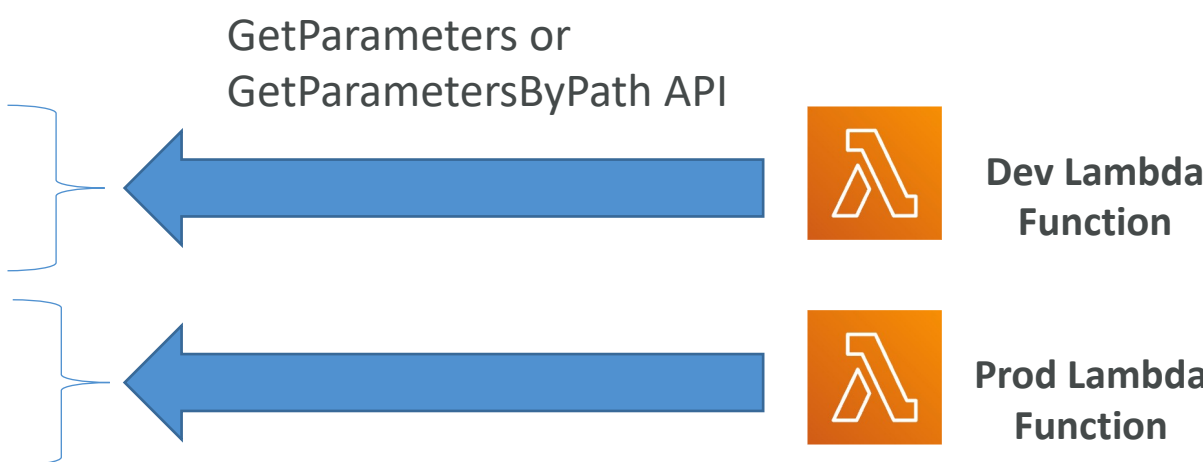
Feature	AWS KMS	AWS CloudHSM
High Availability	AWS Managed Service	Add multiple HSMs over different AZs
Audit Capability	• CloudTrail • CloudWatch	• CloudTrail • CloudWatch • MFA support
Free Tier	Yes	No

SSM Parameter Store

- Secure storage for configuration and secrets
- Optional Seamless Encryption using KMS
- Serverless, scalable, durable, easy SDK
- Version tracking of configurations / secrets
- Security through IAM
- Notifications with Amazon EventBridge
- Integration with CloudFormation



SSM Parameter Store Hierarchy

- /my-department/
 - my-app/
 - dev/
 - db-url
 - db-password
 - prod/
 - db-url
 - db-password
 - other-app/
 - /other-department/
 - /aws/reference/secretsmanager/secret_ID_in_Secrets_Manager
 - /aws/service/ami-amazon-linux-latest/amzn2-ami-hvm-x86_64-gp2 (public)
- 
- GetParameters or
GetParametersByPath API
- Dev Lambda
Function
- Prod Lambda
Function

Standard and advanced parameter tiers

	Standard	Advanced
Total number of parameters allowed (per AWS account and Region)	10,000	100,000
Maximum size of a parameter value	4 KB	8 KB
Parameter policies available	No	Yes
Cost	No additional charge	Charges apply
Storage Pricing	Free	\$0.05 per advanced parameter per month

Parameters Policies (for advanced parameters)

- Allow to assign a TTL to a parameter (expiration date) to force updating or deleting sensitive data such as passwords
- Can assign multiple policies at a time

Expiration (to delete a parameter)

```
{
  "Type": "Expiration",
  "Version": "1.0",
  "Attributes": {
    "Timestamp": "2020-12-02T21:34:33.000Z"
  }
}
```

ExpirationNotification (EventBridge)

```
{
  "Type": "ExpirationNotification",
  "Version": "1.0",
  "Attributes": {
    "Before": "15",
    "Unit": "Days"
  }
}
```

NoChangeNotification (EventBridge)

```
{
  "Type": "NoChangeNotification",
  "Version": "1.0",
  "Attributes": {
    "After": "20",
    "Unit": "Days"
  }
}
```

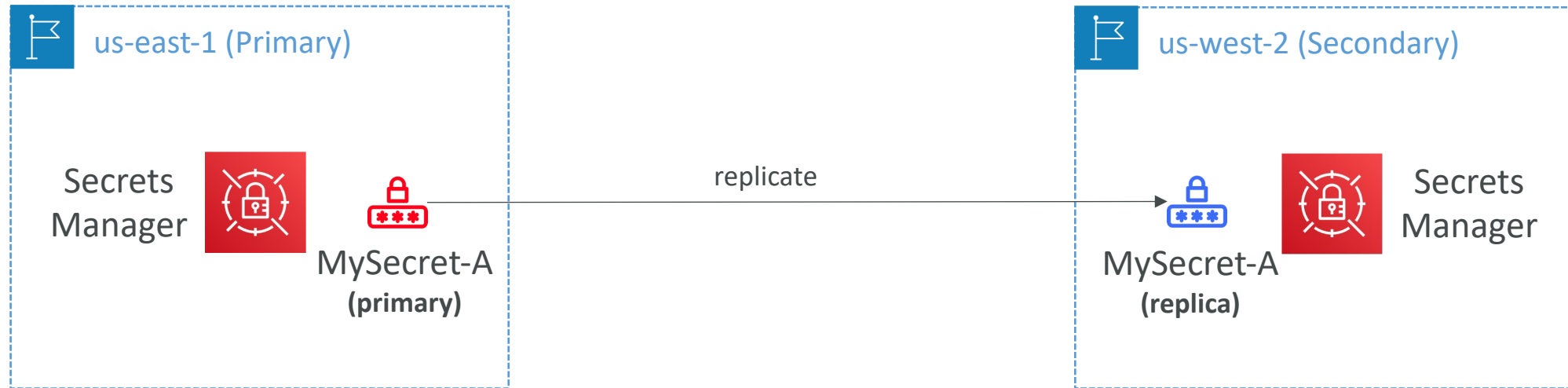


AWS Secrets Manager

- Newer service, meant for storing secrets
- Capability to force **rotation of secrets** every X days
- Automate generation of secrets on rotation (uses Lambda)
- Integration with **Amazon RDS** (MySQL, PostgreSQL, Aurora)
- Secrets are encrypted using KMS
- Mostly meant for RDS integration

AWS Secrets Manager – Multi-Region Secrets

- Replicate Secrets across multiple AWS Regions
- Secrets Manager keeps read replicas in sync with the primary Secret
- Ability to promote a read replica Secret to a standalone Secret
- Use cases: multi-region apps, disaster recovery strategies, multi-region DB...



SSM Parameter Store vs Secrets Manager

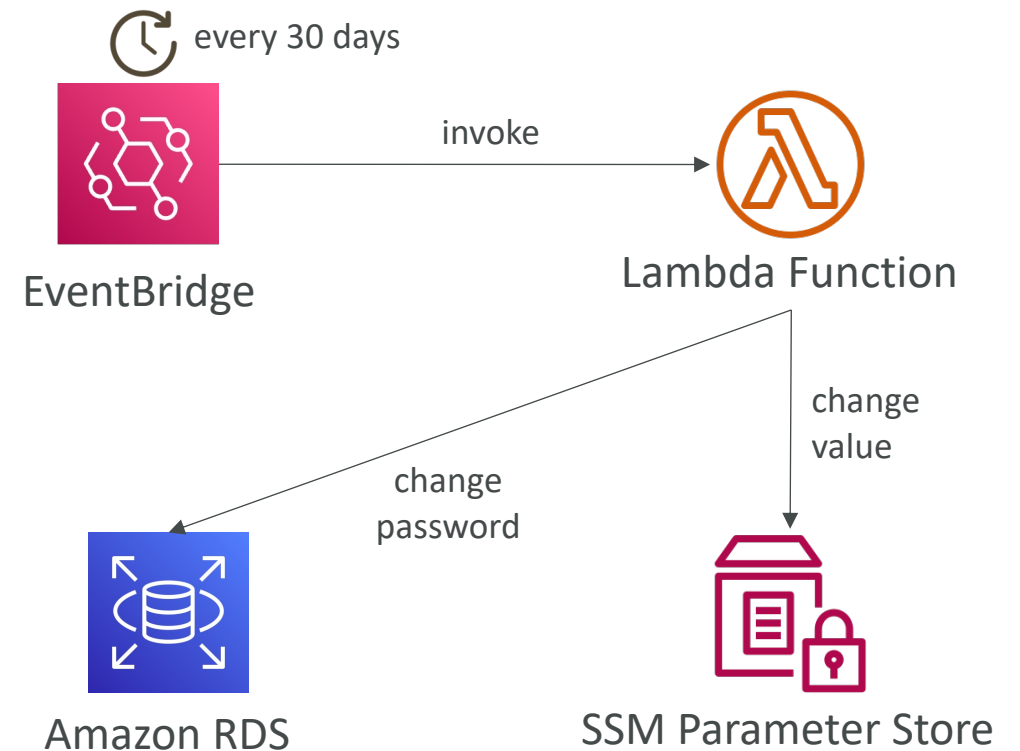
- **Secrets Manager (\$\$\$):**
 - Automatic rotation of secrets with AWS Lambda
 - Lambda function is provided for RDS, Redshift, DocumentDB
 - KMS encryption is mandatory
 - Can integration with CloudFormation
- **SSM Parameter Store (\$):**
 - Simple API
 - No secret rotation (can enable rotation using Lambda triggered by EventBridge)
 - KMS encryption is optional
 - Can integration with CloudFormation
 - Can pull a Secrets Manager secret using the SSM Parameter Store API

SSM Parameter Store vs. Secrets Manager Rotation

AWS Secrets Manager



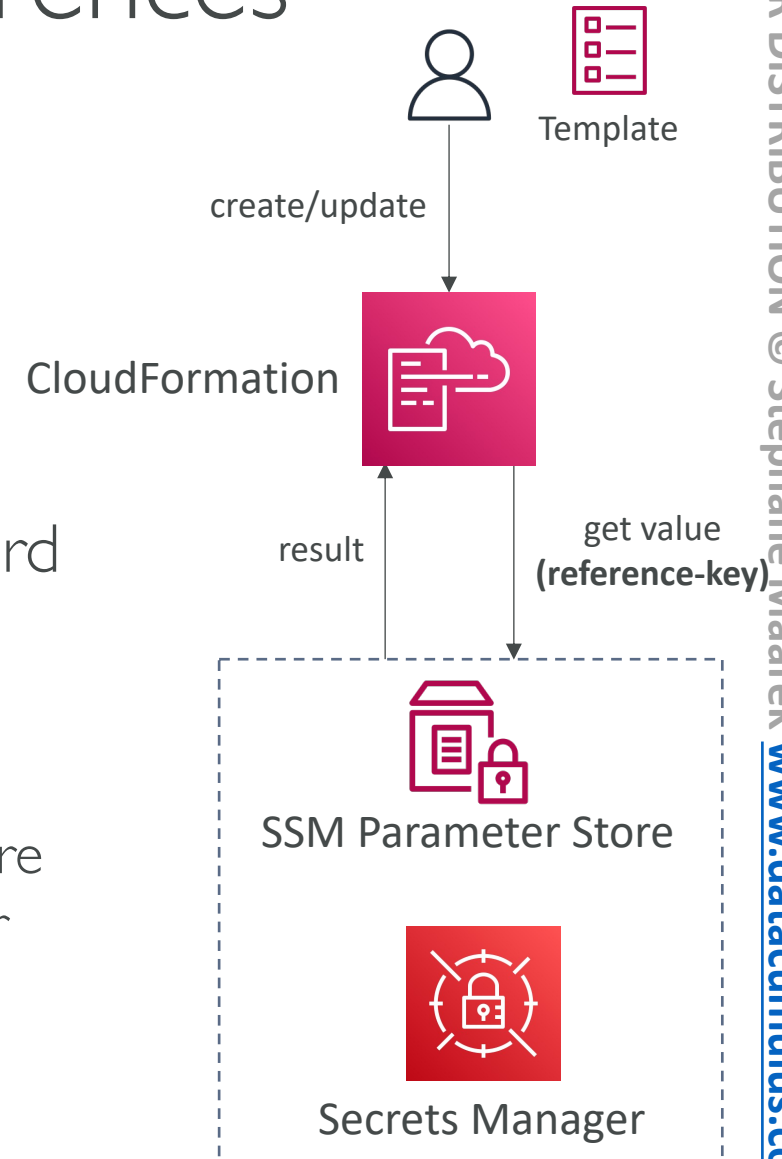
SSM Parameter Store



CloudFormation – Dynamic References

- Reference external values stored in **Systems Manager Parameter Store** and **Secrets Manager** within CloudFormation templates
- CloudFormation retrieves the value of the specified reference during **create/update/delete** operations
- For example: retrieve RDS DB Instance master password from Secrets Manager
- Supports
 - `ssm` – for plaintext values stored in SSM Parameter Store
 - `ssm-secure` – for secure strings stored in SSM Parameter Store
 - `secretsmanager` – for secret values stored in Secrets Manager

`'{{resolve:service-name:reference-key}}'`



CloudFormation – Dynamic References

SSM

```
{{resolve:ssm:parameter-name:version}}
```

Resources:

S3Bucket:

Type: `AWS::S3::Bucket`

Properties:

```
AccessControl: '{{resolve:ssm:S3AccessControl:2}}'
```

SSM Secure

```
{{resolve:ssm-secure:parameter-name:version}}
```

Resources:

IAMUser:

Type: `AWS::IAM::User`

Properties:

UserName: `john`

LoginProfile:

```
Password: '{{resolve:ssm-secure:IAMUserPassword:10}}'
```

Secrets Manager

```
{{resolve:secretsmanager:secret-id:secret-string:json-key:version-stage:version-id}}
```

Resources:

DBInstance:

Type: `AWS::RDS::DBInstance`

Properties:

DBName: `MyRDSInstance`

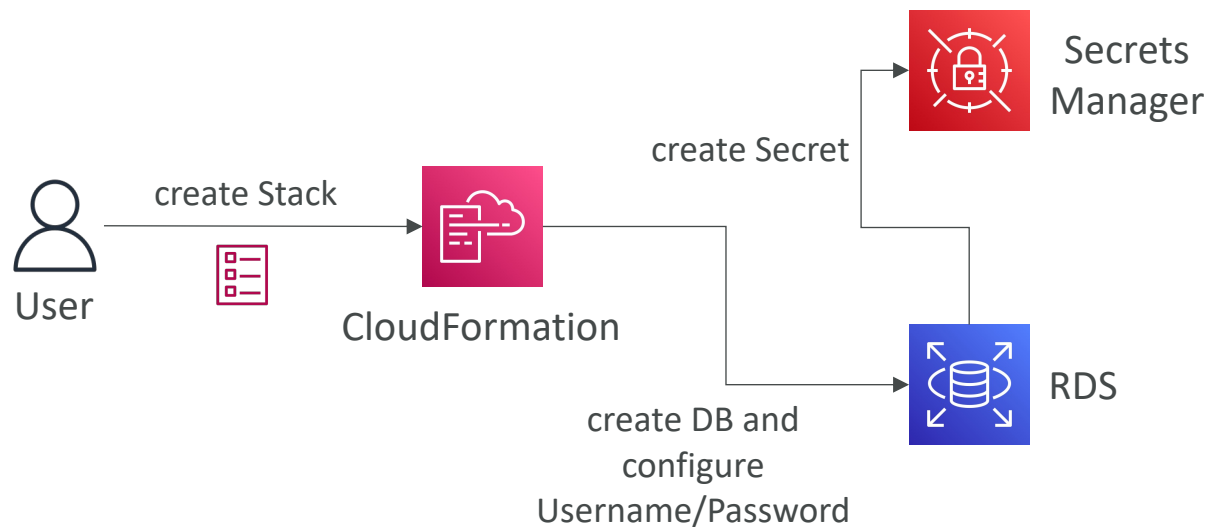
```
MasterUsername: '{{resolve:secretsmanager:MyRDSSecret:SecretString:username}}'
```

```
MasterUserPassword: '{{resolve:secretsmanager:MyRDSSecret:SecretString:password}}'
```

CloudFormation, Secrets Manager & RDS

Option 1 – ManageMasterUserPassword

- `ManageMasterUserPassword` – creates admin secret implicitly
- RDS, Aurora will manage the secret in Secrets Manager and its rotation



Resources:

MyCluster:

Type: `AWS::RDS::DBCluster`

Properties:

Engine: `aurora-mysql`

MasterUsername: `masteruser`

`ManageMasterUserPassword: true`

Outputs:

Secret:

`Value: !GetAtt MyCluster.MasterUserSecret.SecretArn`

CloudFormation, Secrets Manager & RDS

Option 2 – Dynamic Reference

1. secret is generated

Resources:

```
MyDatabaseSecret:
  Type: AWS::SecretsManager::Secret
  Properties:
    Name: MyDatabaseSecret
    GenerateSecretString:
      SecretStringTemplate: '{"username": "admin"}'
      GenerateStringKey: "password"
      PasswordLength: 16
      ExcludeCharacters: '"@/\\"
```

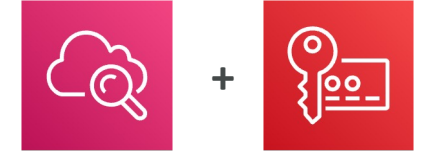
3. link the secret to
RDS DB instance (for rotation)

```
SecretRDSAttachment:
  Type: AWS::SecretsManager::SecretTargetAttachment
  Properties:
    SecretId: !Ref MyDatabaseSecret
    TargetId: !Ref MyDBInstance
    TargetType: AWS::RDS::DBInstance
```

```
MyDBInstance:
  Type: AWS::RDS::DBInstance
  Properties:
    DBName: mydatabase
    AllocatedStorage: 20
    DBInstanceClass: db.t2.micro
    Engine: mysql
```

2. Reference secret in
RDS DB instance

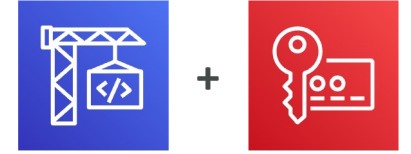
```
MasterUsername: '{{resolve:secretsmanager:MyDatabaseSecret:SecretString:username}}'
MasterUserPassword: '{{resolve:secretsmanager:MyDatabaseSecret:SecretString:password}}'
```



CloudWatch Logs - Encryption

- You can encrypt CloudWatch logs with KMS keys
- Encryption is enabled at the log group level, by associating a CMK with a log group, either when you create the log group or after it exists.
- You cannot associate a CMK with a log group using the CloudWatch console.
- You must use the CloudWatch Logs API:
 - **associate-kms-key** : if the log group already exists
 - **create-log-group**: if the log group doesn't exist yet

CodeBuild Security



- To access resources in your VPC, make sure you specify a VPC configuration for your CodeBuild
- Secrets in CodeBuild:
- Don't store them as plaintext in environment variables
- Instead...
 - Environment variables can reference parameter store parameters
 - Environment variables can reference secrets manager secrets



AWS Nitro Enclaves

- Process highly sensitive data in an isolated compute environment
 - Personally Identifiable Information (PII), healthcare, financial, ...
- Fully isolated virtual machines, hardened, and highly constrained
 - Not a container, not persistent storage, no interactive access, no external networking
- Helps reduce the attack surface for sensitive data processing apps
 - **Cryptographic Attestation** – only authorized code can be running in your Enclave
 - Only Enclaves can access sensitive data (integration with KMS)
- Use cases: securing private keys, processing credit cards, secure multi-party computation...